

Assignment1 - TCPClient.java

```
package tcpchat;

import java.io.*;
import java.net.*;
import java.util.Scanner;
import java.util.logging.*;

public class TCPClient {
    private static final Logger logger = Logger.getLogger(TCPClient.class.getName());

    public static void main(String[] args) {
        String host = "localhost";
        int port = 5000;

        try (Socket socket = new Socket(host, port);
            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            Scanner scanner = new Scanner(System.in)) {

            logger.info("Connected to server at " + host + ":" + port);

            new Thread(() -> {
                try {
                    String line;
                    while ((line = in.readLine()) != null) {
                        System.out.println("[Server]: " + line);
                        if (line.contains("bye")) break;
                    }
                } catch (IOException e) {
                    logger.log(Level.WARNING, "Error reading from server", e);
                }
            }).start();

            while (true) {
                String userInput = scanner.nextLine();
                out.println(userInput);
                if ("bye".equalsIgnoreCase(userInput)) break;
            }

        } catch (IOException e) {
            logger.log(Level.SEVERE, "Client error", e);
        }
    }
}
```

Assignment1 - TCPServer.java

```
package tcpchat;

import java.io.*;
import java.net.*;
import java.util.concurrent.*;
import java.util.logging.*;

public class TCPServer {
    private static final Logger logger = Logger.getLogger(TCPServer.class.getName());
    private static final int PORT = 5000;

    public static void main(String[] args) {
        ExecutorService pool = Executors.newFixedThreadPool(10);

        try (ServerSocket serverSocket = new ServerSocket(PORT)) {
            logger.info("Server started on port " + PORT);

            while (true) {
                Socket clientSocket = serverSocket.accept();
                pool.execute(new ClientHandler(clientSocket));
            }

        } catch (IOException e) {
            logger.log(Level.SEVERE, "Server error", e);
        }
    }

    static class ClientHandler implements Runnable {
        private final Socket socket;

        ClientHandler(Socket socket) {
            this.socket = socket;
        }

        public void run() {
            try {
                BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
                PrintWriter out = new PrintWriter(socket.getOutputStream(), true) {

                    out.println("Enter username:");
                    String username = in.readLine();
                    logger.info(username + " connected from " + socket.getInetAddress());

                    out.println("Welcome, " + username + "! Type 'bye' to exit.");

                    String msg;
                    while ((msg = in.readLine()) != null) {
                        logger.info(username + ": " + msg);
                        out.println("Server Echo: " + msg);
                        if ("bye".equalsIgnoreCase(msg)) break;
                    }
                }
            }
        }
    }
}
```

```
        logger.info(username + " disconnected.");
        socket.close();

    } catch (IOException e) {
        logger.log(Level.WARNING, "Client handler error", e);
    }
}
}
```

Assignment2 - Calculator.java

```
package rmi_calc;

import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Calculator extends Remote {
    int add(int a, int b) throws RemoteException;
    int subtract(int a, int b) throws RemoteException;
    int multiply(int a, int b) throws RemoteException;
    int divide(int a, int b) throws RemoteException;
}
```

Assignment2 - CalculatorImpl.java

```
package rmi_calc;

import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class CalculatorImpl extends UnicastRemoteObject implements Calculator {

    protected CalculatorImpl() throws RemoteException {
        super();
    }

    public int add(int a, int b) throws RemoteException {
        return a + b;
    }

    public int subtract(int a, int b) throws RemoteException {
        return a - b;
    }

    public int multiply(int a, int b) throws RemoteException {
        return a * b;
    }

    public int divide(int a, int b) throws RemoteException {
        return a / b;
    }
}
```

Assignment2 - CalculatorServer.java

```
package rmi_calc;

import java.rmi.Naming;
import java.rmi.registry.LocateRegistry;

public class CalculatorServer {
    public static void main(String[] args) {
        try {
            LocateRegistry.createRegistry(1099);
            System.out.println("RMI registry started...");
            CalculatorImpl obj = new CalculatorImpl();
            Naming.rebind("CalculatorService", obj);
            System.out.println("Calculator Server is ready.");
        } catch (Exception e) {
            System.out.println("Server exception: " + e);
            e.printStackTrace();
        }
    }
}
```

Assignment2 - CalculatorClient.java

```
package rmi_calc;

import java.rmi.Naming;
import java.util.Scanner;

public class CalculatorClient {
    public static void main(String[] args) {
        try {
            Calculator stub = (Calculator) Naming.lookup("rmi://localhost/CalculatorService");

            Scanner sc = new Scanner(System.in);
            System.out.print("Enter first number: ");
            int a = sc.nextInt();
            System.out.print("Enter second number: ");
            int b = sc.nextInt();

            System.out.println("Addition: " + stub.add(a, b));
            System.out.println("Subtraction: " + stub.subtract(a, b));
            System.out.println("Multiplication: " + stub.multiply(a, b));
            System.out.println("Division: " + stub.divide(a, b));

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Assignment3 - Calculator.java

```
package corba_sim;

import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Calculator extends Remote {
    int add(int a, int b) throws RemoteException;
    int subtract(int a, int b) throws RemoteException;
    int multiply(int a, int b) throws RemoteException;
    int divide(int a, int b) throws RemoteException;
}
```


Assignment3 - CalculatorImpl.java

```
package corba_sim;

import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class CalculatorImpl extends UnicastRemoteObject implements Calculator {

    protected CalculatorImpl() throws RemoteException {
        super();
    }

    public int add(int a, int b) throws RemoteException {
        return a + b;
    }

    public int subtract(int a, int b) throws RemoteException {
        return a - b;
    }

    public int multiply(int a, int b) throws RemoteException {
        return a * b;
    }

    public int divide(int a, int b) throws RemoteException {
        return a / b;
    }
}
```

Assignment3 - CalculatorServer.java

```
package corba_sim;

import java.rmi.Naming;
import java.rmi.registry.LocateRegistry;

public class CalculatorServer {
    public static void main(String[] args) {
        try {
            LocateRegistry.createRegistry(1099);
            System.out.println("RMI registry started...");
            CalculatorImpl obj = new CalculatorImpl();
            Naming.rebind("CalculatorService", obj);
            System.out.println("Calculator Server is ready.");
        } catch (Exception e) {
            System.out.println("Server exception: " + e);
            e.printStackTrace();
        }
    }
}
```

Assignment3 - CalculatorClient.java

```
package corba_sim;

import java.rmi.Naming;
import java.util.Scanner;

public class CalculatorClient {
    public static void main(String[] args) {
        try {
            Calculator stub = (Calculator) Naming.lookup("rmi://localhost/CalculatorService");

            Scanner sc = new Scanner(System.in);
            System.out.print("Enter first number: ");
            int a = sc.nextInt();
            System.out.print("Enter second number: ");
            int b = sc.nextInt();

            System.out.println("Addition: " + stub.add(a, b));
            System.out.println("Subtraction: " + stub.subtract(a, b));
            System.out.println("Multiplication: " + stub.multiply(a, b));
            System.out.println("Division: " + stub.divide(a, b));

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Assignment4 - MessageService.java

```
package mpisim;

import java.rmi.Remote;
import java.rmi.RemoteException;

public interface MessageService extends Remote {
    void sendMessage(String sender, String message) throws RemoteException;
}
```

Assignment4 - MessageServiceImpl.java

```
package mpisim;

import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class MessageServiceImpl extends UnicastRemoteObject implements MessageService {

    protected MessageServiceImpl() throws RemoteException {
        super();
    }

    @Override
    public void sendMessage(String sender, String message) throws RemoteException {
        System.out.println("Message received from " + sender + ": " + message);
    }
}
```

Assignment4 - RMIServer.java

```
package mpisim;

import java.rmi.Naming;
import java.rmi.registry.LocateRegistry;

public class RMIServer {
    public static void main(String[] args) {
        try {
            LocateRegistry.createRegistry(1099);
            MessageServiceImpl obj = new MessageServiceImpl();
            Naming.rebind("MessageService", obj);
            System.out.println("MPI-style RMI Server is ready.");
        } catch (Exception e) {
            System.err.println("Server exception: " + e.toString());
            e.printStackTrace();
        }
    }
}
```

Assignment4 - RMIClient.java

```
package mpisim;

import java.rmi.Naming;
import java.util.Scanner;

public class RMIClient {
    public static void main(String[] args) {
        try {
            MessageService stub = (MessageService)
Naming.lookup("rmi://localhost/MessageService");

            Scanner sc = new Scanner(System.in);
            System.out.print("Enter your process name (e.g., Process 1): ");
            String sender = sc.nextLine();

            System.out.print("Enter message to send: ");
            String msg = sc.nextLine();

            stub.sendMessage(sender, msg);

        } catch (Exception e) {
            System.err.println("Client exception: " + e.toString());
            e.printStackTrace();
        }
    }
}
```

Assignment5 - ClockSync.java

```
package berkeley;

import java.util.*;

public class ClockSync {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of slave machines: ");
        int n = sc.nextInt();

        int[] slaveClocks = new int[n];
        System.out.println("Enter the time (in ms) for each slave machine:");
        for (int i = 0; i < n; i++) {
            System.out.print("Slave " + (i + 1) + ": ");
            slaveClocks[i] = sc.nextInt();
        }

        System.out.print("Enter the master's clock time (in ms): ");
        int masterClock = sc.nextInt();

        int sum = masterClock;
        for (int t : slaveClocks) {
            sum += t;
        }

        int avgTime = sum / (n + 1);
        System.out.println("\nAverage time calculated by master: " + avgTime + " ms");

        int masterDiff = avgTime - masterClock;
        System.out.println("Master needs to adjust by: " + masterDiff + " ms");

        for (int i = 0; i < n; i++) {
            int diff = avgTime - slaveClocks[i];
            System.out.println("Slave " + (i + 1) + " needs to adjust by: " + diff + " ms");
        }

        sc.close();
    }
}
```


Assignment6 - TokenRing.java

```
package tokenring;

import java.util.Scanner;

public class TokenRing {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of processes in the ring: ");
        int n = sc.nextInt();

        boolean[] hasToken = new boolean[n];

        System.out.print("Enter the process number (0 to " + (n - 1) + ") that initially holds the token: ");
        int tokenHolder = sc.nextInt();
        hasToken[tokenHolder] = true;

        System.out.print("Enter the number of times to circulate the token: ");
        int rounds = sc.nextInt();

        System.out.println("\nStarting Token Ring Simulation...\n");

        for (int r = 1; r <= rounds; r++) {
            System.out.println("Round " + r + ":");

            for (int i = 0; i < n; i++) {
                if (hasToken[i]) {
                    System.out.println("Process " + i + " has the token and is entering the critical section.");
                    hasToken[i] = false;
                    hasToken[(i + 1) % n] = true;
                    break;
                }
            }

            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }

        sc.close();
        System.out.println("\nToken Ring Simulation Ended.");
    }
}
```

Assignment7 - BullyElection.java

```
package election;

import java.util.Scanner;

public class BullyElection {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of processes: ");
        int n = sc.nextInt();
        int[] processes = new int[n];
        boolean[] alive = new boolean[n];

        System.out.println("Enter process IDs:");
        for (int i = 0; i < n; i++) {
            processes[i] = sc.nextInt();
            alive[i] = true;
        }

        System.out.print("Enter the process ID that detects coordinator failure: ");
        int initiator = sc.nextInt();

        System.out.println("\nElection initiated by process " + initiator);

        int newCoordinator = initiator;

        for (int i = 0; i < n; i++) {
            if (processes[i] > initiator && alive[i]) {
                System.out.println("Process " + initiator + " sends election to " + processes[i]);
                System.out.println("Process " + processes[i] + " replies OK");
                newCoordinator = Math.max(newCoordinator, processes[i]);
            }
        }

        System.out.println("\nProcess " + newCoordinator + " becomes the new coordinator.");
        sc.close();
    }
}
```

Assignment7 - RingElection.java

```
package election;

import java.util.*;

public class RingElection {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of processes in the ring: ");
        int n = sc.nextInt();
        int[] processes = new int[n];

        System.out.println("Enter process IDs:");
        for (int i = 0; i < n; i++) {
            processes[i] = sc.nextInt();
        }

        System.out.print("Enter the process ID that initiates the election: ");
        int initiator = sc.nextInt();

        List<Integer> election = new ArrayList<>();
        int index = -1;

        for (int i = 0; i < n; i++) {
            if (processes[i] == initiator) {
                index = i;
                break;
            }
        }

        System.out.println("\nElection message passing in ring:");
        for (int i = 0; i < n; i++) {
            int idx = (index + i) % n;
            election.add(processes[idx]);
            System.out.println("Process " + processes[idx] + " passes message.");
        }

        int coordinator = Collections.max(election);
        System.out.println("\nProcess " + coordinator + " is elected as new coordinator.");

        sc.close();
    }
}
```

Assignment8 - StudentApiApplication.java

```
package studentapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class StudentApiApplication {
    public static void main(String[] args) {
        SpringApplication.run(StudentApiApplication.class, args);
    }
}
```

Assignment8 - StudentController.java

```
package studentapi;

import org.springframework.web.bind.annotation.*;

import java.util.HashMap;
import java.util.Map;

@RestController
@RequestMapping("/student")
public class StudentController {
    private final Map<Integer, String> studentDB = new HashMap<>();

    @GetMapping("/add")
    public String addStudent(@RequestParam int id, @RequestParam String name) {
        studentDB.put(id, name);
        return "Student added: " + name + " (ID: " + id + ")";
    }

    @GetMapping("/get")
    public String getStudent(@RequestParam int id) {
        String name = studentDB.get(id);
        if (name != null) {
            return "Student ID: " + id + ", Name: " + name;
        } else {
            return "Student not found with ID: " + id;
        }
    }

    @GetMapping("/all")
    public Map<Integer, String> getAllStudents() {
        return studentDB;
    }
}
```

Assignment9 - ChatServer.java

```
package distributedchat;

import java.io.*;
import java.net.*;
import java.util.*;
import java.util.concurrent.*;

public class ChatServer {
    private static final int PORT = 5000;
    private static final Set<ClientHandler> clients = ConcurrentHashMap.newKeySet();

    public static void main(String[] args) {
        System.out.println("Server started on port " + PORT);

        try (ServerSocket serverSocket = new ServerSocket(PORT)) {
            while (true) {
                Socket clientSocket = serverSocket.accept();
                ClientHandler handler = new ClientHandler(clientSocket);
                clients.add(handler);
                new Thread(handler).start();
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    static class ClientHandler implements Runnable {
        private Socket socket;
        private PrintWriter out;
        private String name;

        public ClientHandler(Socket socket) {
            this.socket = socket;
        }

        public void run() {
            try {
                (BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()))) {
                    out = new PrintWriter(socket.getOutputStream(), true);
                    out.println("Enter your name:");
                    name = in.readLine();
                    broadcast(name + " has joined the chat.");

                    String message;
                    while ((message = in.readLine()) != null) {
                        if (message.equalsIgnoreCase("bye")) break;
                        broadcast(name + ": " + message);
                    }
                } catch (IOException e) {
                    e.printStackTrace();
                } finally {
                    try { socket.close(); } catch (IOException ignored) {}
                }
            }
        }
    }
}
```

```
        clients.remove(this);
        broadcast(name + " has left the chat.");
    }
}

void broadcast(String message) {
    for (ClientHandler client : clients) {
        if (client.out != null) {
            client.out.println(message);
        }
    }
}
}
}
```

Assignment9 - ChatClient.java

```
package distributedchat;

import java.io.*;
import java.net.*;
import java.util.Scanner;

public class ChatClient {
    public static void main(String[] args) {
        try (Socket socket = new Socket("localhost", 5000);
            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            Scanner scanner = new Scanner(System.in)) {

            System.out.println(in.readLine()); // "Enter your name:"
            String name = scanner.nextLine();
            out.println(name);

            Thread reader = new Thread(() -> {
                String msg;
                try {
                    while ((msg = in.readLine()) != null) {
                        System.out.println(msg);
                    }
                } catch (IOException e) {
                    System.out.println("Disconnected from server.");
                }
            });
            reader.start();

            while (true) {
                String msg = scanner.nextLine();
                out.println(msg);
                if (msg.equalsIgnoreCase("bye")) break;
            }
        } catch (IOException e) {
            System.out.println("Could not connect to server.");
        }
    }
}
```